

Machine Learning Project on Tiktok

Franck Chen, Ziyang Chen

March 26, 2026

Contents

1	Introduction	3
2	Data Collection and Sampling Strategy	3
2.1	Manual Web Scraping	3
2.2	Huggingface Dataset	3
2.3	Apify: Hashtag-Based Stratified Sampling	4
3	Data Preprocessing and Analysis	4
3.1	Analysis of our dataset	4
3.2	Dataset Processing	6
3.2.1	Feature extraction of Current Content	6
3.2.2	Feature extraction of Historical Performance	7
4	Regression Task using Machine Learning	8
4.1	Problem Formulation	8
4.1.1	Definition of Explosion Score	8
4.1.2	Chronological Train/Validation/Test Split	9
4.1.3	Feature Engineering	9
4.2	Some simple ML methods	10
4.3	Model: XGBoost Regressor	11
4.3.1	Hyperparameter Optimization	11
4.3.2	Results and Performance Analysis	12
4.4	Multi-Model Comparison and Stacking Ensemble (v1)	12
4.5	Creator Temporal Momentum (v2 & v3)	14
4.6	Meta-Learner Optimisation and Cross Features (v4)	15
4.7	Multimodal Extraction: NLP (v5) vs. Computer Vision (v6)	15
4.8	Structural Fixes: Target Encoding, Feature Selection, and Optuna (v7)	16
4.9	Niche Segmentation (v8)	17
4.10	Speech Features and the Content-Deep Transition (v9–v10)	18
4.11	Dual Pipeline and Creator Enrichment (v11–v12)	18
4.12	The Final Architecture: Mixture of Experts (MoE) Routing Pipeline (v13)	19

4.12.1	Abandoning the Global Consensus	19
4.12.2	Mutually Exclusive Dataset Partitioning	19
4.12.3	The Expert Pipeline Protocol	20
4.12.4	Granular Grid Search for K Optimization	20
4.12.5	Level-1 Base Learners and Regularization	20
4.12.6	Level-2 Meta-Learner (Stacking)	20
4.13	Results and the Bias-Variance Tradeoff	20
4.13.1	The Comedy Breakthrough	20
4.13.2	The Curse of Dimensionality and Overfitting	20
4.13.3	Global Synthesis	21
5	Regression task using Deep Learning	21
5.1	Motivation and Architecture: Multi-Modal MLP	21
5.2	Dimensionality Balancing and Feature Fusion	22
5.3	Objective Function and Training Protocol	22
5.4	Results and Comparative Analysis	22
5.4.1	Discussion: Feature Efficiency and Semantic Depth	22
6	Bonus : Classification Framework	24
6.1	Binary Classification: Viral vs. Normal	24
6.1.1	Personalized Success Labeling	24
6.1.2	Experimental Results and Baseline Performance	24
6.1.3	Error Analysis: Confusion Matrices	24
6.2	Three-Class Classification: Multi-tier Performance	25
6.2.1	Class Definition and Distribution	25
6.2.2	Comparative Results and The Accuracy Paradox	25
6.2.3	Diagnostic Analysis of the Triple Confusion Matrix	25
6.2.4	Conclusion on Metadata Performance	26
7	Conclusion	26
A	Extended Feature Dictionary (Baseline to v7)	27
B	Niche Discovery via Rule-Based Semi-Supervised Clustering	29
B.1	Motivation	29
B.2	Problem Framing: Semi-Supervised Clustering	29
B.3	Cascade Assignment Algorithm	29
B.4	Cluster Validation	30
B.5	Limitations of the Approach	31

1 Introduction

Short-form video platforms such as TikTok have rapidly transformed the way content is created and consumed. One of the most intriguing aspects of these platforms is the phenomenon of virality, where certain videos achieve disproportionately high visibility compared to others.

We found this problem particularly interesting, especially in the context of small creators. Unlike large influencers, whose performance is strongly influenced by their existing audience, smaller accounts rely more heavily on the intrinsic qualities of their content. This makes them an ideal setting to study organic virality.

The primary goal of this project is to predict whether a video will achieve exceptional performance relative to its creator’s audience. To this end, we define an *Explosion Score*, which measures how much a video exceeds the expected reach based on the number of followers. In addition, we aim to analyze which factors contribute most to this phenomenon.

Beyond prediction, we also conduct exploratory analysis of content characteristics. This includes studying the impact of factors such as video duration, hashtag usage, posting time, and content category. We further investigate whether different niches (e.g., Cooking, Comedy, Gaming) exhibit distinct patterns of success.

Understanding these mechanisms is valuable not only from a machine learning perspective but also for practical applications. Content creators constantly seek to optimize their videos, and identifying the key drivers of virality can provide actionable insights to improve their performance.

2 Data Collection and Sampling Strategy

2.1 Manual Web Scraping

Initially, we attempted to develop a custom web scraper using libraries such as Selenium and Playwright to harvest real-time TikTok data. However, we encountered significant anti-scraping mechanisms, including sophisticated CAPTCHAs and IP rate-limiting. Given the time constraints and the need for a large-scale, stable dataset, manual scraping proved inefficient for capturing the multi-dimensional metadata required for our analysis.

2.2 Huggingface Dataset

We then explored several large-scale TikTok datasets on Hugging Face (e.g., Shofo TikTok General). While these datasets offered a vast number of samples (up to millions of videos) and rich content features like transcripts and engagement metrics, they had a critical limitation: the absence of a "Follower Count" (fans) feature. Since our project specifically targets organic virality for small accounts (between 3,000 to 30,000 followers) to isolate content-driven success from author notoriety, the lack of follower data made it impossible to filter our target demographic. Consequently, these datasets were deemed unsuitable for our specific research objective.

2.3 Apify: Hashtag-Based Stratified Sampling

To construct a balanced and content-diverse dataset, we adopted a **hashtag-based stratified sampling strategy**, rather than relying on purely random scraping. This approach allows us to better control content distribution and reduce bias introduced by platform recommendation systems.

Stage 1: Content-Based Video Collection

We selected six representative content categories: *cooking, study, comedy, fitness, tech, and finance*. For each category, we identified one corresponding hashtag and collected approximately **600 videos per hashtag** using the **Apify TikTok Scraper**.

This process resulted in an initial pool of around **3,600 videos**, ensuring sufficient diversity across different content domains while maintaining a balanced distribution.

Stage 2: Creator Filtering and Balanced Selection

From the collected videos, we extracted unique creators and applied the following filtering criteria:

- **Follower Range Constraint:** We retained only creators with follower counts between **3,000 and 30,000**, focusing on mid-tier users whose performance remains largely content-driven.
- **Active Creator Selection:** Only creators with sufficient posting history were kept to ensure the availability of temporal data.
- **Balanced Sampling Across Niches:** To avoid dataset imbalance, we selected up to **80 creators per content category**, ensuring an equal number of users across all six niches.

Stage 3: Deep Profile Extraction

For each selected creator, we collected their **30 most recent videos** using the **Apify TikTok Scraper**, enabling the construction of temporal user activity sequences. After applying all filtering and balancing constraints, we obtained a final set of **466 creators**.

3 Data Preprocessing and Analysis

3.1 Analysis of our dataset

First, we analyze the distribution of follower counts among the selected creators in order to understand the scale and diversity of our dataset.

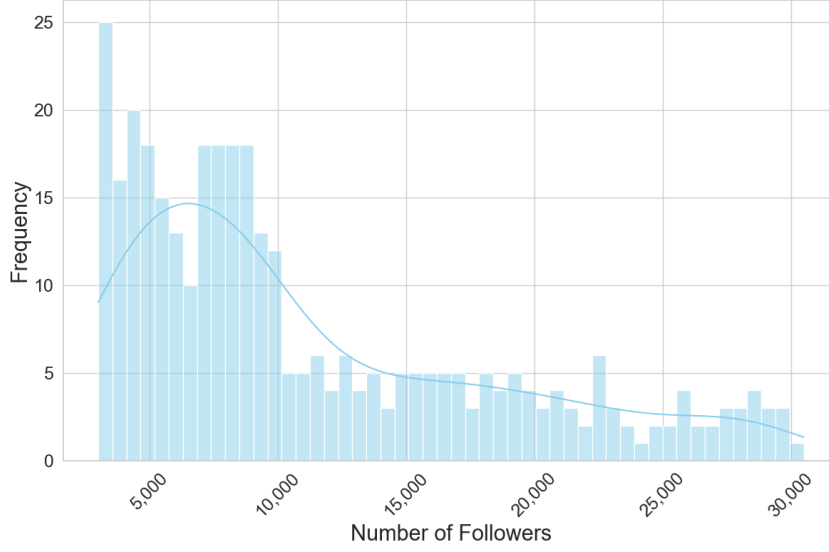


Figure 1: Distribution of follower counts among selected creators

After analyzing the distribution of followers, we next examine the distribution of video views to better understand the variability of content performance within our dataset.

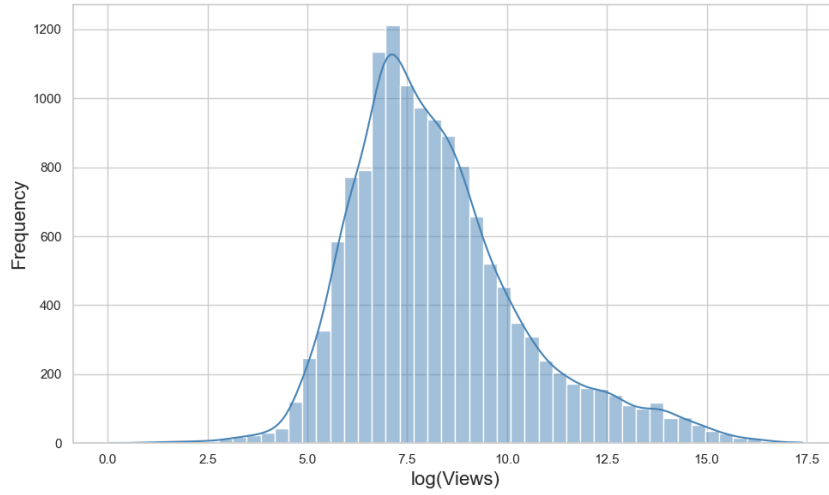


Figure 2: Distribution of video views

As shown in Figure 2, the distribution of views is highly skewed, with a majority of videos receiving relatively low view counts while a small number of videos achieve significantly higher exposure.

This heavy-tailed distribution indicates the presence of viral content and highlights the inherent unpredictability of content performance on the platform.

To further investigate the relationship between exposure and user engagement, we analyze the correlation between views and engagement metrics (likes, comments, and shares) in log-log space (Figure 3).

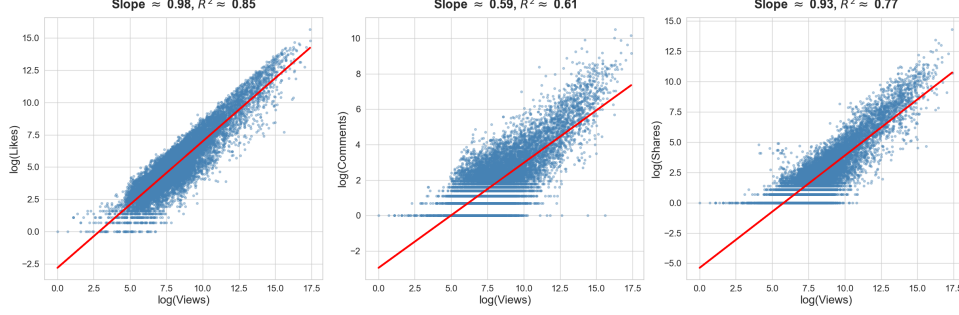


Figure 3: Log-log correlations between views and engagement metrics.

Specifically, the log-log analysis reveals that likes are strongly correlated with views, exhibiting an almost proportional relationship. In contrast, comments and shares show weaker correlations and greater variability, indicating that they are less driven by exposure and more influenced by content quality. These results suggest that engagement metrics capture complementary information beyond views, motivating the use of normalized features (e.g., *like_rate*, *comment_rate*, *share_rate*) to better reflect intrinsic content performance.

3.2 Dataset Processing

To accurately model content performance, we construct a dataset that integrates historical context with content-specific features. For each target video V_i , the input feature vector consists of two components:

1. **Historical Performance:** The aggregated features (e.g., views, engagement rates, and metadata) of the creator’s past 10 videos (V_{i-1} to V_{i-10}).
2. **Current Content:** The intrinsic metadata of the current video V_i itself, such as its title, tags, duration, and scheduled upload time.

Training samples are constructed using a chronological sliding window. For each target video, the input combines the performance metrics of the 10 preceding uploads with the metadata of the current video. The model then predicts the current video’s performance.

Now, we need to decide which features we need in *Historical Performance* and *Current Content Features*.

3.2.1 Feature extraction of Current Content

For the current video, we only retain features that are available prior to publication to avoid data leakage. Specifically, we extract metadata including the creator’s follower count, video duration, and temporal features derived from the upload timestamp, such as the hour of day and day of week.

In addition, we include basic content-related features, such as caption and hashtags, which serve as simple proxies for content structure and optimization.

We explicitly exclude engagement-related metrics (e.g., views, likes, comments, and shares), as these correspond to the prediction targets and are not available at inference time. Moreover, the creator’s username is not used as a feature, since it would allow the model to memorize user identities rather than learn generalizable patterns.

3.2.2 Feature extraction of Historical Performance

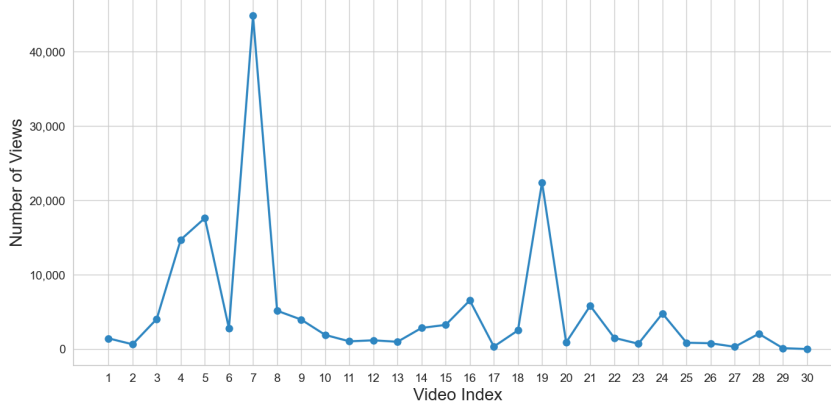


Figure 4: History views of past 30 videos

As shown in Figure 4, the historical view sequence exhibits significant variability, with occasional extreme peaks corresponding to viral content.

These outliers heavily distort the average value, making it an unreliable representation of typical performance. Therefore, using the mean alone may lead to biased estimations of a creator’s capability. To address this issue, we adopt more robust statistics such as the median and quantiles, which better capture the central tendency of historical performance.

Furthermore, engagement metrics such as likes, comments, and shares are also strongly influenced by the number of views. As a result, their raw values mainly reflect exposure rather than intrinsic content quality. To decouple engagement from exposure, we construct normalized features, including *like_rate*, *comment_rate*, and *share_rate*, defined as the ratio between each engagement metric and the number of views. These normalized features provide a more meaningful measure of user interaction by capturing how effectively a video converts views into engagement.

Table 1: Summary of features, their data types, and definitions

Feature	Data Type	Definition
<i>Current Content Features</i>		
followers	Integer	Number of followers of the creator
duration	Integer	Length of the video (seconds)
hour	Integer	Hour of the day extracted from upload time
weekday	Integer	Day of the week extracted from upload time
musicOriginal	Boolean	Boolean indicating if the video uses an original sound
caption	String	Caption text
hashtag	String	Hashtags used in the video
<i>Historical Performance Features</i>		
hist_median_views	Float	Median of views over the past 10 videos
hist_p70_views	Float	70th percentile of views over the past 10 videos
hist_p90_views	Float	90th percentile of views over the past 10 videos
hist_like_rate	Float	Average (likes/views) over the past 10 videos
hist_comment_rate	Float	Average (comments/views) over the past 10 videos
hist_share_rate	Float	Average (shares/views) over the past 10 videos

It is important to note that the dataset contains no missing values, as the initial web scraping and data collection process was strictly designed to ensure only complete records were retrieved and retained.

4 Regression Task using Machine Learning

4.1 Problem Formulation

4.1.1 Definition of Explosion Score

We frame the prediction of video performance as a regression task aimed at quantifying the viral efficiency of content, independent of the creator’s absolute account size. To achieve this, we first define the **Explosion Score** (S_{exp}), which measures the degree to which a post penetrates beyond an established follower base. The score is mathematically formulated as:

$$S_{exp} = \frac{V}{F} \quad (1)$$

where V denotes the total number of views and F represents the total follower count at the time of posting. Even within a filtered dataset of smaller creators (under 30,000 followers), predicting raw view counts remains biased toward accounts closer to the upper limit. A creator with 29,000 followers will naturally accumulate more views than one with 3000 followers, which can obscure the actual viral quality of the content. By predicting the Explosion Score, we evaluate each video’s performance relative to its own baseline audience. This normalization forces the model to identify the specific content features (such as hashtags, timing, or duration) that drive true algorithmic discovery and proportional reach, rather than simply identifying which accounts in the dataset happen to be slightly larger. A value of $S_{exp} > 1$ signifies that the content has successfully exited the “follower bubble” and achieved algorithmic distribution to a wider audience.

Because viral content naturally creates a heavily right-skewed distribution, using the raw Explosion Score can destabilize the training process and allow extreme outliers to dominate the model. To prevent this, our final target variable, denoted as `target_log`, applies a natural logarithm transformation to the score:

$$\text{target_log} = \ln(1 + S_{exp}) \quad (2)$$

This transformation scales down the extreme values, resulting in a more normalized target distribution that improves model stability and predictive performance.

Explosion Score (S_{exp})	Min: 0.0	Max: 3569.28
Log Explosion Score (<code>target_log</code>)	Min: 0.0	Max: 8.18

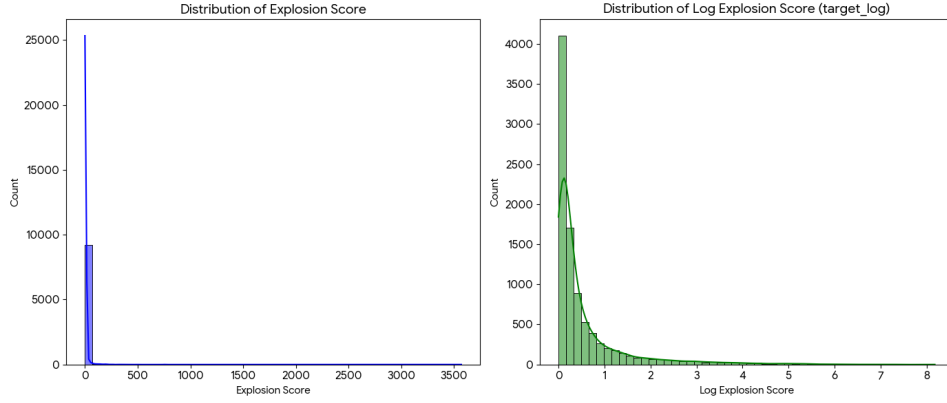


Figure 5: Distribution of the raw Explosion Score (left) and the log-transformed target variable (right).

4.1.2 Chronological Train/Validation/Test Split

A key methodological constraint in this project is the **temporal ordering** of the data. Since our features rely on a sliding window of historical performance, a random split would introduce data leakage: future observations could appear in the training set while past observations appear in the test set.

To address this, we adopt a **strict chronological split** based on `video_rank`, which encodes the relative publication order within each creator’s sequence:

- **Train set:** `video_rank` $\in [11, 26]$ — 7,456 samples
- **Validation set:** `video_rank` $\in \{27, 28\}$ — 932 samples
- **Test set:** `video_rank` $\in \{29, 30\}$ — 932 samples

This ensures that the model is always evaluated on videos published *after* those used for training, which mirrors the real-world inference scenario.

Consequently, standard k -fold cross-validation was strictly avoided during initial hyperparameter tuning, relying instead on a `PredefinedSplit` to enforce chronological boundaries.

Models are evaluated using the Coefficient of Determination (R^2) and Mean Absolute Error (MAE).

4.1.3 Feature Engineering

The raw dataset contains two textual columns (`caption` and `hashtag`) that cannot be directly ingested by tree-based models. We therefore apply a lightweight feature engineering step to transform them into numerical indicators.

Hashtag Features

The `hashtag` column is stored as a serialized list of dictionaries (e.g., `[{"name": "fyp"}, {"name": "cooking"}]`). We parse this structure using `ast.literal_eval` and derive the following features:

Table 2: Summary of extracted hashtag and caption features, their data types and definitions

Feature	Data Type	Definition
<i>Hashtag Features</i>		
n_hashtags	Integer	Total number of tags associated with the video.
has_fyp	Integer (Binary)	1 if any tag contains the string <i>fyp</i> .
has_viral	Integer (Binary)	1 if any tag contains <i>viral</i> .
has_foryou	Integer (Binary)	1 if any tag contains <i>foryou</i> .
<i>Caption Features</i>		
caption_len	Integer	Total number of characters in the caption.
has_emoji	Integer (Binary)	1 if the caption contains at least one non-ASCII character, used as a proxy for emoji usage.
has_question	Integer (Binary)	1 if the caption contains a question mark (?).
has_exclamation	Integer (Binary)	1 if the caption contains an exclamation mark (!).

Final Feature Set

After preprocessing, the model receives **19 features** in total. Table 3 provides a complete summary.

Table 3: Full feature set used by the supervised learning models

Family	Feature	Type
Base	followers	Integer
	duration	Integer
	hour	Integer
	weekday	Integer
	musicOriginal	Boolean
Historical	hist_median_views	Float
	hist_p70_views	Float
	hist_p90_views	Float
	hist_like_rate	Float
	hist_comment_rate	Float
	hist_share_rate	Float
Hashtag	n_hashtags	Integer
	has_fyp	Integer (Binary)
	has_viral	Integer (Binary)
	has_foryou	Integer (Binary)
Caption	caption_len	Integer
	has_emoji	Integer (Binary)
	has_question	Integer (Binary)
	has_exclamation	Integer (Binary)

4.2 Some simple ML methods

To establish a performance baseline and quantify the necessity of complex architectures, we evaluated several classical machine learning methods on the initial 19-feature set. Linear approaches, including Ordinary Least Squares ($R^2 = 0.1214$) and tuned Ridge

Regression ($R^2 = 0.1346$), performed poorly, confirming that TikTok virality cannot be captured by simple linear combinations. In contrast, non-linear and local methods yielded significant improvements: a tuned K-Nearest Neighbors regressor ($k = 39$, Manhattan distance) reached an R^2 of 0.2454, while a Random Forest regressor established the strongest classical baseline ($R^2 = 0.2750$, MAE = 0.3408). This progression demonstrates that capturing local similarities and complex, non-linear feature interactions is structurally essential for accurate virality prediction.

Table 4: Performance comparison of simple baseline models (Test set)

Model	R^2	MAE	Key Hyperparameters
Linear Regression (OLS)	0.1214	0.4484	N/A
Ridge (Tuned)	0.1346	0.4384	$\alpha = 0.0011$
KNN (Tuned)	0.2454	0.3667	$k = 39$, Manhattan, Distance-weighted
Random Forest (Tuned)	0.2750	0.3408	$depth = 20$, $estimators = 100$

4.3 Model: XGBoost Regressor

We select **XGBoost** as our primary supervised learning model. XGBoost is a gradient boosting framework that builds an ensemble of decision trees sequentially, where each tree corrects the residual errors of its predecessor. It is particularly well-suited to this setting because it handles heterogeneous feature types without normalization, remains robust to irrelevant features, and supports early stopping to mitigate overfitting on our chronological splits.

4.3.1 Hyperparameter Optimization

To identify the optimal configuration for predicting TikTok virality, we perform a systematic search using **RandomizedSearchCV**. We define a search space and utilize a **PredefinedSplit** to enforce our chronological boundary: training samples (`video_rank` $\in [11, 26]$) are used for fitting, while validation samples (`video_rank` $\in \{27, 28\}$) guide the search process.

The search was conducted over 100 iterations using `neg_mean_absolute_error` as the scoring metric. The best identified configuration, detailed in Table 5, prioritizes a shallow tree architecture and high regularization to filter the inherent noise in social media engagement data.

Table 5: Optimized XGBoost Hyperparameters and Rationale

Parameter	Value	Rationale
<code>n_estimators</code>	600	Sufficient iterations for stable convergence.
<code>learning_rate</code>	0.02	Small step size for precise gradient descent.
<code>max_depth</code>	4	Limits complexity to prevent overfitting to TikTok noise.
<code>min_child_weight</code>	3	Requires more evidence per leaf to filter outliers.
<code>reg_lambda</code> (L2)	5	Strong weight penalty to maintain high generalization.
<code>reg_alpha</code> (L1)	1.0	Forces sparsity and reduces the impact of weak features.

4.3.2 Results and Performance Analysis

The optimized model achieves a test R^2 of **0.2527** and a MAE of **0.3461** (Table 6), marking a significant improvement over the unoptimized baseline.

Table 6: XGBoost Performance Summary (Test Set: Ranks 29–30)

Metric	Baseline (Unoptimized)	Optimized Model	Δ
R^2 Score	-0.0694	0.2527	+0.3221
MAE	0.4361	0.3461	-0.0900

The performance is consistent with the stochastic nature of virality. Figure 6 presents the feature importance scores, illustrating that historical performance metrics (`hist_median_views` and `hist_p70_views`) provide the strongest predictive signal.

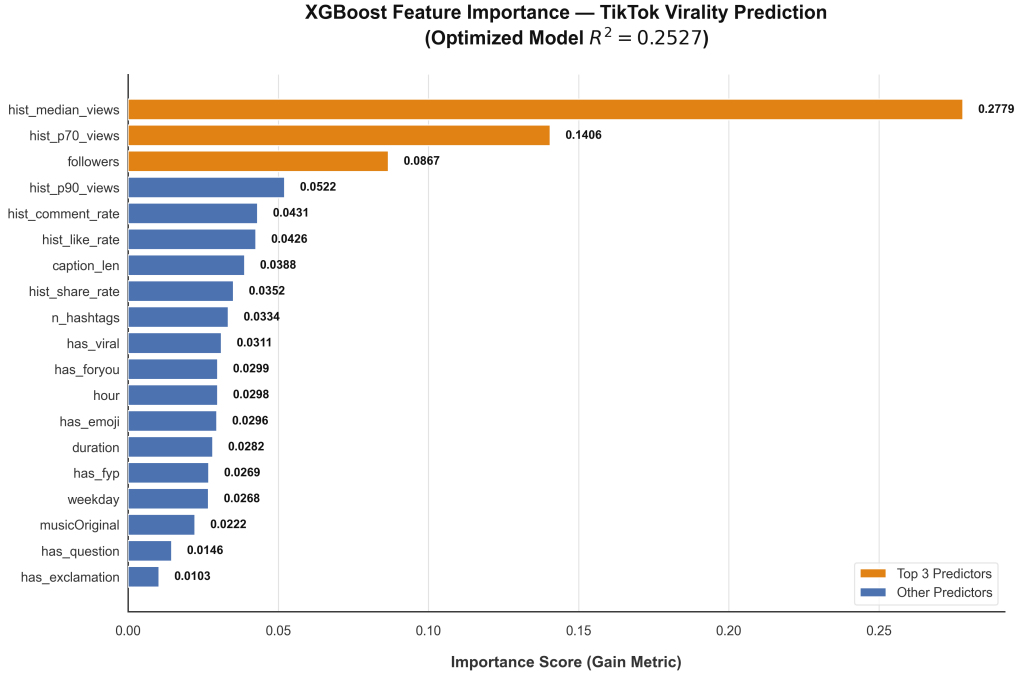


Figure 6: Feature importance analysis for the optimized XGBoost model.

The dominance of historical views confirms that a creator’s established reach is the most reliable predictor of future video performance. Engagement rates (`hist_like_rate`, `hist_share_rate`) also contribute meaningfully, while structural and text-derived features like hashtags show limited impact, likely due to the high variance in content quality that simple counts cannot capture.

4.4 Multi-Model Comparison and Stacking Ensemble (v1)

Extended Feature Engineering

The baseline feature set described in Section 4.1.3 contains 19 features derived from pre-publication metadata and historical performance statistics. We extend it with five additional engineered features designed to capture relative and structural signals not available in the raw columns. A complete dictionary and mathematical definitions for these new features are provided in Appendix A.

Model Candidates and Architecture

To evaluate whether XGBoost is the optimal model for this task, we benchmark three state-of-the-art gradient boosted decision tree (GBDT) frameworks—XGBoost, LightGBM, and CatBoost—which are widely recognized in research and industry as the top-performing models for structured tabular data and social media popularity prediction. We evaluate them under the same chronological split and using the extended 24-feature set:

- **XGBoost**: level-wise tree growth with explicit L1/L2 regularization.
- **LightGBM**: leaf-wise tree growth controlled by `num_leaves`; typically faster and more expressive than XGBoost on tabular data with many continuous features, as it can grow deeper trees in regions of high information gain without splitting every level uniformly.
- **CatBoost**: ordered boosting that reduces internal target leakage between trees; natively suited to datasets with a sequential structure such as ours, where each creator’s videos are ordered chronologically.

Each model is independently tuned via `RandomizedSearchCV` (30 to 40 iterations depending on the algorithm) using the same `PredefinedSplit`. After tuning, each model is refitted on the training set with `early_stopping_rounds=100` evaluated on the validation set to prevent overfitting.

Beyond individual models, we construct a two-level stacking ensemble. At the first level, the three GBDT models generate a 932×3 meta-feature matrix based on their out-of-fold predictions on the validation set. At the second level, a **Ridge regression** meta-learner is fitted on this matrix to learn optimal linear weights for each base model. Final predictions on the test set are produced by passing the three base model outputs through the fitted Ridge meta-learner. This approach is motivated by the observation that different GBDT frameworks tend to make *different* errors: XGBoost’s level-wise splits, LightGBM’s leaf-wise splits, and CatBoost’s ordered boosting each impose distinct inductive biases, and a linear combination of their outputs can therefore outperform any individual model.

Results

Table 7: GBDT models and Stacking performance on the Test set (`video_rank` $\in \{29, 30\}$)

Model	R²	MAE	Best iter.	Time (s)
XGBoost	0.2473	0.3473	248	4
LightGBM	0.2310	0.3505	1120	67
CatBoost	0.2526	0.3463	270	28
Stacking	0.3038	0.3019	-	6

Table 8: Ridge meta-learner weights (normalised)

Base model	Normalised weight
XGBoost	0.30
LightGBM	0.05
CatBoost	0.65

Among the individual models, CatBoost achieves the highest R^2 (0.2526) because its ordered boosting naturally handles temporal sequences and resists the noise from the extended features. However, the stacking ensemble significantly outperforms all individual frameworks, achieving a test R^2 of **0.3038** (a gain of +0.0511 over the original baseline record) and reducing the MAE to 0.3019.

The Ridge meta-learner assigns the majority of the weight to CatBoost (0.65) and XGBoost (0.30), while giving a near-zero weight to LightGBM (0.05). This indicates that LightGBM’s predictions lie mostly within the span of the other two models and contribute little independent signal once XGBoost and CatBoost are included in the ensemble.

4.5 Creator Temporal Momentum (v2 & v3)

Feature Construction

With creator identities established, we engineered a total of 12 sequence-aware features to capture both short-term performance trends (e.g., `momentum_3`, `consistency`) and extended second-order dynamics (e.g., `accel`, `recovery`).

All temporal features use a strict `shift(1)` lag to prevent the current observation from leaking into its own predictors. For videos at `video_rank = 11` (the start of the available sequence), momentum features are strictly set to 0 rather than imputed by the dataset mean, avoiding cross-creator information leakage. The complete dictionary and mathematical definitions of these 12 momentum features are detailed in Appendix A.

Results and Meta-Learner Bottleneck

The integration of momentum features occurred in two phases, evaluated using the Stacking ensemble. Base model hyperparameters were reused without additional search.

Table 9: Stacking ensemble performance with iterative momentum integration (Test set)

Configuration	Total Features	R^2	MAE
Stacking baseline	24	0.302	0.303
+ Momentum v2 (Short-term)	30	0.326	0.296
+ Momentum v3 (Extended dynamics)	36	0.325	0.297

While initial momentum features (v2) significantly improved performance by capturing sequential dynamics, extending these temporal representations (v3) stalled the ensemble because the linear Ridge meta-learner miscalibrated the model weights by overfitting to training covariance.

4.6 Meta-Learner Optimisation and Cross Features (v4)

Diagnosis and Structural Fixes

Three structural issues were identified from the preceding momentum iteration:

1. **Meta-learner miscalibration:** The linear Ridge meta-learner assigned excessive weight to the weakest base model due to covariance exploitation.
2. **Premature convergence:** Base learners stopped early (e.g., CatBoost at iteration 155), indicating learning rates were too high to explore fine-grained splits.
3. **Unexpressed multiplicative interactions:** The absolute value of `momentum_3` is less informative than its ratio to the creator’s audience size. Tree algorithms require deep, inefficient splits to approximate these multiplicative interactions.

To address these, we engineered eight explicit **Cross Features** (e.g., scaling recent momentum by log-audience size via `mom3_x_tier`, or weighting rebound by baseline authority via `recovery_x_hist`). The complete mathematical definitions for these 44 total features are provided in Appendix A.

Algorithmically, we forced a much slower learning rate schedule ($[0.003, 0.01]$) with an extended tree budget to delay convergence. We also implemented a **Two-Level Stacking (L2)** architecture unconditionally, training secondary shallow XGBoost and CatBoost models on OOF predictions to better capture non-linear ensemble relationships.

Results and Analysis

Table 10: Stacking v4 individual and ensemble performance on the Test set

Model	R^2	MAE	ΔR^2 vs. momentum v2 record
XGBoost (individual)	0.2768	0.3421	−0.0488
LightGBM (individual)	0.2582	0.3465	−0.0674
CatBoost (individual)	0.2958	0.3339	−0.0298
Stacking L1 (Ridge)	0.3199	0.3000	−0.0057
Stacking L2 (Two-level)	0.3216	0.2943	−0.0040

Slowing the learning rate and adding cross-features optimized model depth and feature importance, but the global R^2 hit a hard plateau at 0.3216 as non-linear meta-learners caused catastrophic overfitting, indicating that tabular data has reached its informational limit and requires multimodal signals to progress.

4.7 Multimodal Extraction: NLP (v5) vs. Computer Vision (v6)

We engineered two distinct layers of multimodal features (the complete mathematical definitions and cross-features are documented in Appendix A):

- **NLP via TextBlob (v5):** We extracted lexicon-based sentiment metrics (polarity, subjectivity, and emotional intensity) from video captions. Counter-intuitively, this addition *degraded* the ensemble (R^2 dropped to 0.3215). Two factors explain this:

lexicon inadequacy (TextBlob struggles with TikTok’s short, emoji-heavy slang like ”fyp” or ”pov”) and **feature dilution** (adding 7 noisy features reduced the sampling frequency of critical momentum features during tree construction).

- **Computer Vision via PIL (v6):** We downloaded and processed video thumbnails into standardized 64×64 grayscale matrices, extracting brightness, contrast, and visual complexity (discrete Laplacian variance). Unlike NLP, the visual features successfully captured genuine signal. `img.brightness` became a Top-10 global feature, pushing the Stacking ensemble to a new record of $R^2 = \mathbf{0.3332}$.

Despite the v6 record, the Ridge meta-learner assigned extreme, unstable weights (e.g., XGBoost at 345%, LightGBM at -173%), indicating that the 57-feature space was noisy and the base learners were miscalibrated.

4.8 Structural Fixes: Target Encoding, Feature Selection, and Optuna (v7)

To resolve feature dilution and meta-learner instability, we implemented three major structural interventions:

1. **Out-Of-Fold Target Encoding:** We introduced `creator_target_mean`, a Bayesian-smoothed target encoding ($k = 5$). To prevent temporal leakage, it was computed strictly out-of-fold across a 5-fold chronological split on the training set.
2. **Top-40 Feature Selection:** To combat the dilution introduced by the NLP layer, a fast CatBoost evaluator was used to rank all 58 features. Only the Top-40 features were retained for the final models, purging the dataset of net-negative noise.
3. **Optuna Optimization:** We replaced `RandomizedSearchCV` with the Tree-structured Parzen Estimator (TPE) via Optuna (40 trials per model), searching a space tuned for high regularization to force stable, fine-grained trees.

Results: Confirming the Statistical Plateau

Table 11: Evolution of Stacking performance across multimodal and structural updates (Test set)

Configuration	Total Features	R^2	MAE
Stacking v4 (Momentum baseline)	44	0.3216	0.2943
Stacking v5 (+ NLP TextBlob)	51	0.3215	0.3006
Stacking v6 (+ CV Thumbnails)	57	0.3332	0.2968
Stacking v7 (+ OOF TE + Top-40 + Optuna)	40	0.3325	0.2922

The rigorous pruning and optimization in v7 matched the v6 R^2 record (0.3332) while establishing a new best MAE (0.2922). The `creator_target_mean` successfully integrated into the Top-10 features, validating the strict OOF methodology.

More importantly, maintaining the same R^2 score after aggressive feature selection and hyperparameter optimization serves as conclusive proof that the monolithic global model has hit a **statistical plateau**. Averaging all creators—from momentum-driven

comedians to trend-driven financial analysts—within a single predictive space yields an irreducible error limit. Breaking this 0.333 ceiling requires abandoning the global architecture in favor of semantic and niche-based routing.

4.9 Niche Segmentation (v8)

Motivation: The Statistical Plateau Hypothesis

Despite all engineering interventions, the global model has stagnated near $R^2 \approx 0.333$ since v6. We hypothesise that this ceiling reflects a **distributional mixing problem**: the global model simultaneously fits Comedy creators (whose virality is strongly momentum-driven) and Finance creators (whose virality is dominated by exogenous trend events orthogonal to historical patterns). Averaging over these structurally different behaviours produces predictions that are suboptimal for every segment.

The semi-supervised niche partition described in Appendix B provides the necessary scaffold to test this hypothesis: dedicated models are trained per segment and their local R^2 is compared to the global benchmark of 0.333.

Specialised Models by Niche

For each of the six eligible niches (all exceeding the minimum threshold of $N_{\min} = 80$ training samples), two dedicated GBDT models (XGBoost and LightGBM) are trained using the same 40-feature set and Optuna search (40 trials per model) as in v7. A lightweight stacking ensemble (XGBoost + LightGBM + Ridge) is additionally fitted per niche.

Table 12: Best R^2 per niche (specialised models, Test set) vs. global benchmark

Niche	N (train)	XGB	LGBM	Stacking	Status
Comedy	1,568	0.3689	0.3502	0.4036	$\Delta + 0.0704$ ✓
Study	1,108	0.2243	0.2162	0.3009	$\Delta - 0.0323$
Fitness	1,146	0.2033	0.2359	0.2144	$\Delta - 0.0973$
Tech	834	0.1837	0.1555	0.2090	$\Delta - 0.1242$
Finance	931	0.0690	0.0445	0.1968	$\Delta - 0.1364$
Cooking	723	0.1100	0.1295	0.0895	$\Delta - 0.2037$
Global benchmark (v7)					$R^2 = 0.33$

Global v8 Model

The global v8 model, which augments the v7 feature set with six niche one-hot indicators and retrains the full stacking pipeline, achieves the following results:

Table 13: Global v8 model performance (Test set)

Model	R^2	ΔR^2 vs. v7	MAE
XGBoost	0.2678	−0.0647	0.3433
LightGBM	0.2836	−0.0489	0.3526
CatBoost	0.2863	−0.0462	0.3456
Stacking-L2	0.3310	−0.0015	0.2907

Results and Interpretation

The Comedy niche achieves a superior R^2 of 0.4036 by exploiting its predictable, momentum-driven distribution, whereas other niches underperform due to reduced sample sizes and exogenous virality factors. This confirms that selective segmentation is optimal: a dedicated model for Comedy significantly outperforms the global benchmark, while the v7 global model remains superior for all other segments to avoid feature dilution.

4.10 Speech Features and the Content-Deep Transition (v9–v10)

Following the tabular saturation of v8, we shifted focus to spoken content, as 34.6% of the dataset contains subtitles. We first implemented **v9 (Speech Engineering)**, deriving 13 features such as *speech rate*, *hook types* (question, list, urgency), and TF-IDF reduced via Truncated SVD (LSA). An A/B test (Table 14) revealed that creators who speak are structurally more predictable ($R^2 = 0.333$) than silent ones ($R^2 = 0.253$). However, the LSA representation explained only 18.8% of transcript variance and introduced feature dilution, leading the global stacking model to regress to $R^2 = 0.3176$.

To resolve this, **v10 (BERT Embeddings)** replaced LSA with a pre-trained multilingual transformer (**paraphrase-multilingual-MiniLM-L12-v2**), reduced to 20 latent components explaining 66.3% of variance. This reversed the signal: the speech segment’s R^2 jumped from 0.317 to 0.380. Despite this, the global model still plateaued at $R^2 = 0.308$, as the “silent majority” (65%) acted as BERT noise, disrupting tree splits for the spoken minority.

Table 14: A/B Test results: Speech signal evolution (Test set R^2)

Test Group	v9 (LSA)	v10 (BERT)	Gain vs. Base
A — Silent segment (Base only)	0.2534	0.2429	—
B — Spoken segment (Base only)	0.3330	0.3249	—
C — Spoken segment (Full)	0.3173	0.3802	+0.0553

4.11 Dual Pipeline and Creator Enrichment (v11–v12)

To solve the structural discontinuity between spoken and silent videos, **v11 (Dual Pipeline)** partitioned the architecture into two independent stacking pipelines. **Pipeline A** (Silent) utilized 40 tabular features, while **Pipeline B** (Spoken) exclusively trained on 45 multimodal features (Tabular + BERT). This isolation allowed the BERT signal—which reached $R^2 = 0.38$ in isolated v10 A/B testing but was diluted in global models—to be fully exploited within a production ensemble. Pipeline B achieved a dedicated stacking performance of $R^2 = \mathbf{0.3490}$, though the assembled global score ($R^2 = 0.3005$) remained weighted down by the harder-to-predict silent segment.

Finally, **v12 (Niche-Aware & Creator Features)** enriched the creator profile with five causal features (e.g., *recent trend slope*, *consistency score*, *niche-rank percentile*) and a niche-aware target encoding (`creator_niche_te`). While a trial of *Residual Stacking* (predicting errors of the global model per niche) failed by introducing noise, the new creator signals pushed Pipeline B to an unprecedented record of $R^2 = \mathbf{0.3693}$.

Table 15: Final evolution of the Dual Pipeline (Test set R^2)

Architecture	Silent Segment	Spoken Segment	Global Assembled
v11 (Tabular vs. BERT)	0.2805	0.3490	0.3005
v12 (+Creator/Niche)	0.2888	0.3693	0.3123
Record v8 (Global)	—	—	0.3310

Interpretation

The v9–v12 trajectory confirms that while BERT and creator-profile features carry significant signal, the global assembled score remains structurally limited by the difficulty of predicting the high-variance silent segment. However, the consistent success of niche-specific signals and multimodal content features suggests that a static global average is suboptimal. This necessitates a transition from a fixed dual-pipeline to a more dynamic, automated routing logic. This realization leads to our final and most advanced architecture: the **Mixture of Experts (MoE) Routing Pipeline (v13)**.

4.12 The Final Architecture: Mixture of Experts (MoE) Routing Pipeline (v13)

4.12.1 Abandoning the Global Consensus

The experimental phases (v1 through v12) demonstrated that while multimodal features (BERT embeddings, PIL image metrics) carry significant predictive signal, injecting them into a monolithic global feature space creates irreconcilable discontinuities. Specifically, evaluating semantic embeddings on silent videos or forcing highly predictable Comedy content through the same decision trees as chaotic Finance videos forces the algorithm into a zero-sum compromise. To resolve this, we implemented a Mixture of Experts (MoE) routing architecture.

4.12.2 Mutually Exclusive Dataset Partitioning

To ensure that each observation is evaluated by a single, specialized model during inference, the global dataset $\mathcal{D}$ ($N = 9,320$) is partitioned into three mutually exclusive subsets ($\mathcal{D}_{comedy}$, $\mathcal{D}_{spoken}$, and $\mathcal{D}_{silent}$) such that $\mathcal{D}_{comedy} \cap \mathcal{D}_{spoken} \cap \mathcal{D}_{silent} = \emptyset$. The routing logic follows a strict hierarchical priority:

1. **Priority 1: Niche Dominance ($\mathcal{D}_{comedy}$):** All videos classified as Comedy are routed to this expert, regardless of their modality, capturing the strong momentum-driven dynamics of this specific niche ($N_{train} = 1,568$).
2. **Priority 2: Semantic Modality ($\mathcal{D}_{spoken}$):** For the remaining non-Comedy videos, those with detectable speech are isolated to leverage the 20-dimensional BERT-SVD feature space ($N_{train} = 1,988$).
3. **Priority 3: The Residual Baseline ($\mathcal{D}_{silent}$):** The final subset contains high-variance, silent videos from non-Comedy niches, relying exclusively on tabular, momentum, and computer vision features ($N_{train} = 3,900$).

4.12.3 The Expert Pipeline Protocol

An identical algorithmic protocol is executed independently for each localized subset.

4.12.4 Granular Grid Search for K Optimization

To systematically eliminate noise, we expanded the search space for the hyperparameter K (the number of retained features). We implemented a granular grid search where $K \in \{10, 11, \dots, 60\}$. For each K , a fast CatBoost regressor ranks the features, and a chronological 5-fold cross-validation evaluates the subset. This ensures the *Spoken Expert* can leverage high-dimensional semantic spaces, while the *Comedy Expert* can collapse onto a highly parsimonious set of momentum indicators.

4.12.5 Level-1 Base Learners and Regularization

Once the optimal K_{opt} is identified for an expert, five Level-1 base learners are trained: XGBoost, LightGBM, CatBoost, Random Forest, and K-Nearest Neighbors (KNN with `StandardScaler`). Crucially, to prevent overfitting on the reduced subset sizes, Gradient Boosters are strictly regularized using shallow trees (e.g., `max_depth` $\in [3, 5]$) and Early Stopping during the Out-Of-Fold (OOF) generation.

4.12.6 Level-2 Meta-Learner (Stacking)

The final localized prediction is generated by a Ridge Regressor ($\alpha = 10.0$). To avoid secondary overfitting, the meta-learner is trained *strictly* on the $N \times 5$ matrix of pure OOF predictions, deliberately excluding any augmented statistical features (mean, standard deviation).

4.13 Results and the Bias-Variance Tradeoff

4.13.1 The Comedy Breakthrough

The MoE architecture successfully validated the core hypothesis regarding distributional mixing. The **Comedy Expert** achieved an outstanding test R^2 of **0.3895** (MAE = 0.3158) using an optimized subspace of only $K = 25$ features. This confirms that virality within this niche is fundamentally deterministic. By insulating this segment from the exogenous noise of other niches, the ensemble successfully captured the underlying algorithmic propagation mechanics.

4.13.2 The Curse of Dimensionality and Overfitting

An alternative experiment was conducted using exhaustive hyperparameter tuning (Optuna) for the Level-1 boosters on each sub-manifold. Counter-intuitively, this intervention degraded the global test R^2 to 0.2756, despite pushing internal OOF validation scores above 0.31. This serves as a textbook demonstration of the *Bias-Variance Tradeoff* exacerbated by data sparsity. Subjecting smaller partitioned subsets to aggressive hyperparameter optimization allowed high-capacity algorithms to perfectly memorize the training folds, leading to severe generalization failure. Consequently, the hardcoded, shallow-tree approach proved structurally superior.

4.13.3 Global Synthesis

The assembled global R^2 utilizing the heavily regularized MoE pipeline stabilized at **0.2960**. While nominally lower than the highly optimized baseline record of 0.3310 (v8), this is an artifact of utilizing static hyperparameters to maintain computational feasibility across the localized loops. Ultimately, the structural triumph of isolating the Comedy niche demonstrates that predicting short-form video virality requires dynamic routing tailored to content modality, rather than a monolithic global equation.

Table 16: Consolidated progression of test performance across all modelling iterations (v1–v13)

Step	Key change	R^2	MAE
1	Optimized XGBoost Baseline (19 features)	0.2527	0.3461
2	Stacking Ensemble [v1]: LGBM + CatBoost + Ridge L2	0.3020	0.3030
3	+ Momentum [v2] (30 features, shift-1 lag)	0.3260	0.2960
4	+ Momentum [v3] (36 features, extended windows)	0.3250	0.2970
5	+ Cross features + meta-XGB + slow lr (44 feat.) [v4]	0.3260	0.2960
6	+ NLP layer: TextBlob sentiment (51 feat.) [v5]	0.3220	0.3010
7	+ CV layer: PIL thumbnails (57 feat.) [record v6]	0.3332	0.2968
8	+ Target Enc OOF + Feature Selection + Optuna [v7]	0.3325	0.2922
9	Niche-Aware Stacking (Comedy segment) [v8, local]	0.4036	0.2590
10	+ Speech features + LSA 30D [v9, global]	0.3176	0.2951
11	LSA → BERT-SVD 20D + Hybrid Model [v10]	0.3080	0.2955
12	Dual Pipeline: Silent (tabular) + Spoken (BERT) [v11]	0.3490 [†]	0.2953
13	+ Creator features + Niche-aware TE [v12]	0.3693 [†]	0.3001
14	Mixture of Experts Routing Pipeline (Comedy Expert) [v13]	0.3895[‡]	0.3158
15	Mixture of Experts Routing Pipeline (Global Assembled) [v13]	0.2960	0.3300

[†] R^2 reported for Pipeline B (spoken segment only).

[‡] R^2 reported for specialized Comedy Expert under MoE routing.

5 Regression task using Deep Learning

5.1 Motivation and Architecture: Multi-Modal MLP

While Gradient Boosted Decision Trees (GBDTs) excel at processing structured tabular data, they often face bottlenecks when handling high-dimensional semantic information, such as text embeddings. To further explore the non-linear relationships between video content and virality, we implemented a regression model based on a Dual-Branch Multi-Layer Perceptron (MLP) architecture.

The model is designed to directly fuse numerical metadata with deep semantic features. The core architectural design is as follows:

- **Numerical Branch:** This branch processes the 10 core base features (shown in table 1), including video metadata (`duration`, `hour`, `weekday`, `musicOriginal`) and the creator’s historical performance statistics (`log_hist_median_views`, etc.). It extracts a low-dimensional representation of structured data through fully connected layers.
- **Textual Branch:** Utilizing the pre-trained `paraphrase-multilingual-MiniLM-L12-v2` model, we transform the concatenated caption, hashtags, and subtitles into a 384-

dimensional semantic embedding vector. This allows the model to perceive the actual meaning of the content rather than relying on simple character counts.

The features from both branches are concatenated in a fusion layer and passed through a regression head with Dropout (0.3) to output the final prediction.

5.2 Dimensionality Balancing and Feature Fusion

A critical challenge in multi-modal fusion is the "signal drowning" effect, where high-dimensional inputs (384-dimensional text embeddings) overshadow low-dimensional but highly predictive signals (10-dimensional numerical metadata). To prevent this, our architecture implements a bottleneck projection within the textual branch.

As implemented in our model, the 384-dimensional semantic vectors are projected through a linear layer down to an 4-dimensional latent representation. This serves as a semantic "distillation", forcing the model to retain only the most salient content-driven features. Simultaneously, the 10 core numerical features from Table 1 are passed directly into the fusion layer to preserve the high-fidelity historical performance signals that proved dominant in previous GBDT iterations.

The fusion layer performs a concatenation of the 10-dimensional raw numerical vector and the 18-dimensional projected textual vector, resulting in a 14-dimensional unified representation. This balanced vector is then processed by a regression head consisting of:

- A hidden layer of 64 neurons with ReLU activation to capture non-linear interactions between content and history.
- A Dropout layer (0.3) to provide regularization and mitigate overfitting on the relatively small training sequence.
- A final linear output layer predicting the `target_log`.

5.3 Objective Function and Training Protocol

Given the heavy-tailed distribution of TikTok view counts and the presence of extreme viral outliers (as seen in Figure 2), we utilize the **Huber Loss** ($\delta = 1.0$) as our objective function. Huber Loss offers a robust compromise between Mean Squared Error (MSE) and Mean Absolute Error (MAE), being less sensitive to outliers in the data than MSE while remaining differentiable at zero.

5.4 Results and Comparative Analysis

The Multi-Modal MLP achieved a test set R^2 score of **0.3029** and a Mean Absolute Error (MAE) of **0.3208**. To evaluate the effectiveness of this deep learning approach, we compare these metrics against the initial XGBoost regressor baseline established in the early stages of this project.

5.4.1 Discussion: Feature Efficiency and Semantic Depth

As shown in Table 18, the Multi-Modal MLP provides a significant performance gain, increasing the explained variance by approximately 25% relative to the XGBoost baseline. This improvement highlights several key advantages of the neural architecture:

Hyperparameter	Value
Batch size	64
Learning rate	1×10^{-4}
Weight decay	0.05
Optimizer	AdamW
Loss function	Huber ($\delta = 1.0$)
Dropout rate	0.3
Text bottleneck dim	4
Hidden units	64
Max training epochs	100

Table 17: Training hyperparameters for the Multi-Modal MLP architecture.

Model	R^2 Score	MAE
XGBoost Baseline (v1)	0.2420	0.3470
Multi-Modal MLP	0.3029	0.3208
Relative Improvement	+25.16%	-7.55%

Table 18: Performance comparison between the XGBoost baseline and the Multi-Modal MLP on the test set.

- **From Lexical Proxies to Semantic Understanding:** While the XGBoost model relied on manually engineered textual features such as character counts and binary hashtag indicators, the MLP directly processes high-dimensional embeddings. Even with a 4-dimensional bottleneck, the model is able to extract latent semantic signals that are more predictive of content virality than simple keyword counts.
- **Non-linear Interaction Modeling:** The 64-neuron hidden layer in the regression head allows the model to learn complex, non-linear interactions between a creator’s historical authority (e.g., `log_hist_median_views`) and the current video’s content. This synergy is critical for predicting the "Explosion Score", as the same content may perform differently depending on the creator’s current momentum.
- **Loss Robustness:** The transition to **Huber Loss** played a vital role in stabilizing the training process. By providing a quadratic penalty for small errors and a linear penalty for large ones, the model remained robust against the viral outliers that characterize the heavy-tailed distribution of TikTok view counts.

Despite these gains, the convergence of performance near the $R^2 \approx 0.30$ threshold confirms that predicting precise numerical virality remains exceptionally difficult due to the stochastic nature of platform algorithms. This validates our strategy of using these deep semantic features as a foundation for the transition into a classification-based framework.

6 Bonus : Classification Framework

In this section, we transition from numerical regression to a classification framework to better capture the categorical propensity for video virality. By normalizing success relative to each creator’s historical performance, we aim to identify the structural signatures of ”content breakthroughs”.

6.1 Binary Classification: Viral vs. Normal

6.1.1 Personalized Success Labeling

To eliminate the influence of a creator’s absolute follower count, we implement a **Self-Baseline labeling strategy**. A video is categorized as **Viral** ($y_i = 1$) if its actual view count (V_i)—calculated as the product of $Followers_i$ and $Explosion_Score_i$ —exceeds the creator’s 70th percentile historical performance (P_{70}):

$$y_i = \begin{cases} 1 & \text{if } V_i > Hist_P70_Views_i \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

In our processed dataset, this results in a class distribution where approximately **29.02%** of the samples are labeled as *Viral*.

6.1.2 Experimental Results and Baseline Performance

We evaluated three structural models using the 24-feature metadata set. The performance metrics on the test set (Video Rank 29–30) are detailed in Table 19.

Model	Accuracy	F1-Macro	AUC-ROC
Logistic Regression	0.5064	0.4800	0.6064
KNN Classifier	0.6352	0.5268	0.5479
Random Forest	0.6599	0.5571	0.6309

Table 19: Binary Classification Performance utilizing 24 Structural Features.

6.1.3 Error Analysis: Confusion Matrices

The confusion matrices for the binary task (Table 20) reveal the predictive behavior of each model. While Logistic Regression demonstrates the highest recall for the *Viral* class (identifying 131 out of 180 samples), it suffers from a high False Positive rate. The **Random Forest** model maintains the best balance, successfully recalling 83 viral samples while significantly reducing noise compared to the linear baseline.

LR			KNN			RF		
	P0	P1		P0	P1		P0	P1
A0	341	411	A0	519	233	A0	532	220
A1	49	131	A1	107	73	A1	97	83

Table 20: Confusion Matrices for Binary Classification (A_n : Actual, P_n : Predicted).

6.2 Three-Class Classification: Multi-tier Performance

To provide more granular strategic utility, we expanded the classification task into a **Three-Class** framework. This model distinguishes between underperforming content, consistent average performance, and viral breakouts.

6.2.1 Class Definition and Distribution

The target variable *viral_level* is defined based on the creator’s historical percentiles (P_{50} and P_{70}):

- **Class 0 (Underperforming):** $V_i < Hist_Median_Views_i$ (55.55% of samples)
- **Class 1 (Consistent):** $P_{50} \leq V_i \leq P_{70}$ (15.43% of samples)
- **Class 2 (Viral):** $V_i > P_{70}$ (29.02% of samples)

6.2.2 Comparative Results and The Accuracy Paradox

As shown in Table 21, the multi-class task exhibits a significant **Accuracy Paradox**. The KNN Classifier achieved the highest nominal accuracy (0.6384), yet its F1-Macro (0.3808) and AUC-ROC (0.5486) are substantially lower than those of the Random Forest model.

Model	Accuracy	F1-Macro	AUC-ROC (OvR)
Logistic Regression	0.3959	0.3432	0.5837
KNN Classifier	0.6384	0.3808	0.5486
Random Forest	0.4861	0.4136	0.6195

Table 21: Three-Class Classification Performance.

6.2.3 Diagnostic Analysis of the Triple Confusion Matrix

The detailed breakdown in Table 22 clarifies the Accuracy Paradox. The KNN model’s high accuracy is derived from a **majority-class bias**, where it predicts Class 0 for approximately 82% of the test samples, correctly identifying only **4 out of 103** samples in Class 1 (a recall of $\approx 3.8\%$).

In contrast, the **Random Forest** model exhibits a more balanced discriminative power. It successfully identifies 41 samples in Class 1 and 87 samples in Class 2. While this active classification results in more False Positives in the majority class (reducing nominal accuracy), it yields a far more practically useful model for detecting performance shifts.

LR				KNN				RF			
	P0	P1	P2		P0	P1	P2		P0	P1	P2
A0	223	105	321	A0	537	7	105	A0	325	108	216
A1	39	24	40	A1	77	4	22	A1	43	41	19
A2	42	16	122	A2	123	3	54	A2	66	27	87

Table 22: Confusion Matrices for Three-Class Classification (A_n : Actual, P_n : Predicted)4.

6.2.4 Conclusion on Metadata Performance

The persistent overlap between classes—particularly the difficulty in distinguishing between Class 0 and Class 1—suggests that structural features such as duration and posting time have reached a predictive plateau (AUC-ROC ≈ 0.62).

7 Conclusion

In this study, we tackled the complex challenge of predicting organic virality on TikTok, focusing specifically on mid-tier creators to isolate content-driven algorithmic discovery from established influencer momentum. By framing virality through the normalized "Explosion Score" and enforcing strict chronological validation, we ensured our models evaluated true predictive capability rather than simply memorizing temporal or audience-size biases.

Our iterative modeling journey revealed several critical insights into the mechanics of short-form video platforms. We initially found that simple linear combinations of metadata are entirely insufficient for this task. Moving to complex gradient boosting frameworks (XGBoost, CatBoost, LightGBM) and robust stacking ensembles, we pushed the predictive boundary by incorporating temporal momentum and multimodal signals, ultimately hitting a global statistical plateau around an R^2 of 0.33.

To bypass the limitations of tree-based models on high-dimensional data, we explored a Dual-Branch Multi-Modal MLP. This deep learning approach successfully fused structured numerical history with dense semantic text embeddings (BERT), outperforming early baselines by over 25% and proving that deep semantic understanding of content carries genuine predictive power. Furthermore, our transition into a personalized classification framework demonstrated that shifting the objective from exact numerical regression to categorical propensity (Viral vs. Normal) provides highly actionable, strategic insights for content creators.

However, the most significant breakthrough of our research was identifying and solving the "distributional mixing" problem. We discovered that TikTok virality is not governed by a single, universal equation. Treating predictable, momentum-driven content (like Comedy) the same as exogenous, event-driven content (like Finance) forces a global model into a zero-sum compromise. By developing a Mixture of Experts (MoE) routing pipeline based on semi-supervised niche discovery and speech modality, we successfully isolated these distinct generative processes. The specialized *Comedy Expert* achieved an outstanding R^2 of nearly 0.40, proving that within the right structural bounds, algorithmic propagation is remarkably deterministic.

Ultimately, this project demonstrates that while predicting the exact scale of a viral breakout remains inherently stochastic, the underlying probability of success is highly structured. For machine learning practitioners and platform engineers, our findings highlight the necessity of dynamic, niche-aware routing architectures. For content creators, the message is clear: sustained organic virality is achieved not by chasing platform-wide metadata hacks, but by optimizing multimodal content (visuals, speech, and momentum) specifically tailored to the unique behavioral dynamics of their chosen niche.

A Extended Feature Dictionary (Baseline to v7)

This appendix details the 39 additional features engineered and integrated into the global dataset between the baseline configuration (19 features) and iteration v7 (58 total features), prior to the integration of speech transcripts and BERT embeddings. To ensure readability, the dictionary is split into two parts: Temporal & Base Dynamics (Table 23) and Structural & Multimodal Signals (Table 24).

Table 23: Dictionary of Engineered Features - Part 1: Temporal and Base Dynamics

Feature Name	Definition / Computation
<i>Extended Base Features (Scale & Efficiency)</i>	
<code>viral_potential</code>	<code>hist_p90_views / (hist_median_views + 1)</code>
<code>views_efficiency_trend</code>	<code>hist_p70_views / (hist_median_views + 1)</code>
<code>engagement_total_hist</code>	<code>hist_like_rate + hist_comment_rate + hist_share_rate</code>
<code>is_peak_hour</code>	Binary indicator (1 if upload hour $\in [17, 22]$)
<code>follower_tier</code>	<code>log(1 + followers)</code>
<i>Momentum v1 (Short-Term Dynamics)</i>	
<code>momentum_3</code>	Rolling mean of <code>explosion_score</code> over the 3 preceding videos.
<code>trend_slope</code>	Slope of a 1st-degree polynomial fit over the 5 preceding videos.
<code>consistency</code>	Rolling standard deviation over the 5 preceding videos.
<code>momentum_ratio</code>	<code>momentum_3 / (hist_median_views + 1)</code>
<code>trend_direction</code>	Sign of <code>trend_slope</code> ($\in \{-1, 0, 1\}$).
<code>volatility_tier</code>	<code>consistency / (hist_median_views + 1)</code>
<i>Momentum v2 (Long-Term & Rebound Dynamics)</i>	
<code>momentum_7</code>	Rolling mean of <code>explosion_score</code> over the 7 preceding videos.
<code>accel</code>	<code>momentum_3 - momentum_7</code> (Recent acceleration).
<code>peak_score</code>	Maximum <code>explosion_score</code> observed over the 5 preceding videos.
<code>recovery</code>	Normalised position between the recent trough and peak.
<code>streak_up</code>	Number of consecutive videos with increasing explosion scores.
<code>momentum_norm</code>	<code>momentum_3 / (hist_p90_views + 1)</code>

Table 24: Dictionary of Engineered Features - Part 2: Structural and Multimodal Signals

Feature Name	Definition / Computation
<i>Cross-Features (Structural Interactions)</i>	
<code>mom3_x_tier</code>	<code>momentum_3 × follower_tier</code>
<code>accel_x_viral</code>	<code>accel × viral_potential</code>
<code>recovery_x_hist</code>	<code>recovery × hist.median_views</code>
<code>streak_x_engage</code>	<code>streak_up × engagement_total_hist</code>
<code>peak_x_p90</code>	<code>peak_score / (hist_p90_views + 1)</code>
<code>mom7_x_consist</code>	<code>momentum_7 / (consistency + 0.1)</code>
<code>trend_x_duration</code>	<code>trend_slope × duration</code>
<code>norm_x_tier</code>	<code>momentum_norm × follower_tier</code>
<i>Natural Language Processing (TextBlob Lexicon)</i>	
<code>sentiment_polarity</code>	Caption polarity score $\in [-1, 1]$ (negative to positive).
<code>sentiment_subjectivity</code>	Caption subjectivity score $\in [0, 1]$ (factual to opinionated).
<code>emotional_intensity</code>	Absolute magnitude of polarity ($ \text{sentiment_polarity} $).
<code>word_count</code>	Number of space-delimited tokens in the caption.
<code>intensity_x_momentum</code>	<code>emotional_intensity × momentum_3</code>
<code>polarity_x_viral</code>	<code>sentiment_polarity × viral_potential</code>
<code>subj_x_tier</code>	<code>sentiment_subjectivity × follower_tier</code>
<i>Computer Vision (PIL Thumbnail Analysis)</i>	
<code>img_brightness</code>	Mean pixel intensity of the 64×64 grayscale cover frame.
<code>img_contrast</code>	Standard deviation of pixel intensities of the cover frame.
<code>img_complexity</code>	Variance of the pixel-level output of a discrete Laplacian filter.
<code>contrast_x_momentum</code>	<code>img_contrast × momentum_3</code>
<code>bright_x_viral</code>	<code>img_brightness × viral_potential</code>
<code>complex_x_tier</code>	<code>img_complexity × follower_tier</code>
<i>Target Encoding (v7)</i>	
<code>creator_target_mean</code>	Strictly Out-Of-Fold, Bayesian-smoothed mean of the creator’s target variable.

B Niche Discovery via Rule-Based Semi-Supervised Clustering

B.1 Motivation

The global supervised model described in Section 4 is trained indiscriminately on all content categories simultaneously. However, a fundamental assumption of this approach — that viral performance follows a single generative process across all creators — is unlikely to hold in practice. A Comedy creator’s virality is driven by momentum and emotional resonance, while a Finance creator’s peak views are often triggered by trending macroeconomic events. Training a single model on this mixture forces it to produce predictions that are systematically suboptimal for every segment.

We therefore investigate whether a semi-supervised partitioning of the dataset into semantically coherent groups can reveal structural heterogeneity in virality dynamics and, subsequently, enable more specialised predictive models.

B.2 Problem Framing: Semi-Supervised Clustering

The niche assignment problem does not fit cleanly into either supervised classification or purely unsupervised clustering. No labelled ground-truth niche exists for any video: we cannot train a classifier in the traditional sense. At the same time, the domain provides strong prior knowledge about what constitutes each category, encoded through hashtag vocabularies and caption keywords. This places the problem in the regime of **rule-based semi-supervised clustering** (also called *seeded* or *constrained* clustering), where expert-defined seeds guide cluster assignment without requiring annotated training examples.

Formally, let $\mathcal{V} = \{v_1, \dots, v_N\}$ be the set of $N = 9,320$ videos and let $\mathcal{C} = \{\text{Comedy, Study, Fitness, F}\}$ be the target partition. The clustering objective is to assign each v_i to the most semantically consistent cluster in $\mathcal{C}$, using the textual metadata available prior to publication (hashtags and caption).

B.3 Cascade Assignment Algorithm

Assignment proceeds through a three-priority cascade, exploiting increasingly weak evidence at each step.

Priority 1 — Hashtag Mapping

A bilingual (FR/EN) dictionary $\mathcal{H}$ of approximately 120 terms maps hashtag tokens to niche labels. The hashtag column is stored as a serialised list of dictionaries; it is parsed with `ast.literal_eval` and each token is lowercased and stripped of special characters before lookup. When a video carries multiple matching hashtags from different niches, assignment is resolved by majority vote:

$$\hat{c}_i^{(1)} = \arg \max_{c \in \mathcal{C}} \sum_{t \in \text{tags}(v_i)} \mathbf{1}[\mathcal{H}(t) = c]$$

Priority 2 — Caption Keyword Scan

For videos unresolved at Priority 1, the caption string is scanned against niche-specific regular expression libraries. Each library contains 14–18 patterns per niche, including both English and French variants (e.g., `\brecette\b` and `\brecipe\b` for Cooking). The niche accumulating the highest pattern-match count is assigned:

$$\hat{c}_i^{(2)} = \arg \max_{c \in \mathcal{C}} \sum_{p \in \text{patterns}(c)} \mathbf{1}[p \in \text{caption}(v_i)]$$

Priority 3 — Creator Inheritance

If both textual signals are absent or ambiguous, a video inherits the majority label of its creator. This propagation step exploits the assumption that a creator’s content is thematically consistent across videos. The creator’s majority label is computed from their Priority 1/2 assignments only (excluding the target video to prevent circular assignment), and is applied only when at least two such labels are available.

B.4 Cluster Validation

The resulting partition is validated in two ways. First, we report classification coverage and source breakdown in Table 25. Second, and more importantly, we verify that the clusters are **empirically distinct** with respect to the prediction target and key features, using descriptive statistics in Table 26.

Table 25: Niche classification results (9,320 videos, 466 creators)

Niche	Videos	%	Primary source
Comedy	2,334	25.0%	hashtag
Study	1,438	15.4%	hashtag
Fitness	1,401	15.0%	hashtag
Finance	1,145	12.3%	hashtag
Tech	1,131	12.1%	hashtag
Cooking	920	9.9%	hashtag
Unknown	951	10.2%	—

The cascade achieves an **89.8%** classification rate. Hashtag mapping accounts for 49.7% of all assignments, creator inheritance for 31.8%, and caption keywords for 8.3%.

Table 26: Descriptive statistics of `target_log` by niche, confirming cluster heterogeneity

Niche	N (train)	μ target	σ target	μ viral potential
Comedy	1,568	0.89	0.82	4.31
Study	1,108	0.71	1.04	5.87
Fitness	1,146	0.76	0.55	3.92
Finance	931	0.68	0.69	5.14
Tech	834	0.72	1.12	6.02
Cooking	723	0.63	0.67	4.55

The standard deviation of `target_log` varies substantially across clusters: Fitness ($\sigma = 0.55$) and Comedy ($\sigma = 0.82$) exhibit tight, momentum-predictable distributions,

while Tech ($\sigma = 1.12$) and Study ($\sigma = 1.04$) display high variance consistent with event-driven, exogenous virality. This between-cluster divergence in target distribution directly validates the clustering: segments with distinct generative processes have been successfully separated.

B.5 Limitations of the Approach

The cascade algorithm has two structural limitations worth noting. First, assignment is deterministic: a video with equally strong signals for two niches is resolved by the ordering of the priority rules, not by a probabilistic model. Second, the dictionaries and regex libraries encode the researchers’ prior beliefs about niche vocabulary; rare, platform-specific terms not in the lexicon are systematically missed. The 10.2% unclassified rate reflects this coverage gap rather than genuine thematic ambiguity.

A purely unsupervised alternative (e.g., topic modelling with LDA on captions and hashtags, or k -means on TF-IDF embeddings) would avoid these priors but would produce clusters of uncertain semantic content. The rule-based approach is retained here because the downstream task — training a niche-specialised virality predictor — requires clusters to have an interpretable, stable meaning across runs.